

Norrin — Trustworthy Process Monitor: piano di attacco

2026-09-18 · @Someone

Verdetto: fattibile, e vincibile senza essere del campo

Questa challenge non premia l'accuratezza del modello. Premia l'architettura della fiducia. È la notizia migliore per un team al primo hackathon.

Il dataset è il Tennessee Eastman Process, il benchmark più studiato al mondo per fault detection industriale. Classificare i suoi guasti è un problema risolto dal 2015: chiunque con un gradient boosting arriva al 95% di accuratezza in mezz'ora. Norrin lo sa. Per questo la rubrica non contiene la parola "accuracy" nemmeno una volta.

Contiene invece questa frase, che è il cuore di tutto:

Un'inferenza a bassa confidenza con ragionamento visibile deve contare più di un'etichetta sicura senza ragionamento.

E una gate condition non negoziabile: **i dati grezzi non lasciano mai l'ambiente dell'operatore**. Un team che manda righe del CSV dentro un prompt è squalificato di fatto, per quanto bella sia la demo.

Cosa decide davvero il punteggio

Fattore	Peso reale	Chi vince
Accuratezza del rilevamento fault	Bassa	Nessuno: tutti ci arrivano
Tracciabilità evidenza → conclusione	Molto alta	Chi mostra il perché di ogni numero
Separazione dato rotto vs processo rotto	Alta, criterio a sé	Chi la dimostra con un caso live
Controlli operatore reali e funzionanti	Alta	Chi ha un override che cambia lo stato
Portabilità su dati non-sensore	Alta	Chi gira su un secondo dataset
Data minimization	Gate: passa o fallisci	Chi lo prova, non chi lo dichiara

Il vostro vero rischio non è tecnico. È di scope. Gli otto deliverable sono tanti e in 5 persone, senza contratti di interfaccia definiti nella prima ora, finite con cinque pezzi che non si parlano alle 3 di notte. La sezione sui contratti è la parte più importante di questo documento.

Il vostro vantaggio. Non sapere nulla di processi chimici è quasi un bonus: la challenge chiede esplicitamente di non usare conoscenza pregressa come scorciatoia. Chi conosce il TE è tentato di scrivere le etichette a mano, e quella è la strada che perde punti su "end-to-end autonomy".

Cosa chiede la challenge

Costruire un agente autonomo che prende un CSV industriale senza documentazione e ne esce con una diagnosi che un operatore può contestare. Tutto il resto sono corollari di questa frase.

Il problema narrativo: un sensore va fuori taratura per settimane restando dentro il range atteso, quindi nessun allarme scatta. Quando la qualità del prodotto crolla, la causa è sepolta in dashboard e log. Stessa forma nei dati aziendali: migliaia di errori di inserimento manuale che singolarmente passano ogni validazione.

Gli otto deliverable

Sono tassativi. La presentazione li elenca come griglia 4x2 e dice *"build these eight, and show the evidence behind each"*.

#	Deliverable	Cosa deve esistere concretamente
1	Sensor understanding report	Ruolo inferito per ogni colonna, con l'evidenza statistica e un livello di confidenza
2	Automated data quality checks	Completeness, validity, consistency, timeliness + check compilati da regole in linguaggio naturale
3	Drift and anomaly detection	Deviazione nel tempo, flag riferito al segnale responsabile
4	Root-cause diagnosis	Tipo di fault, sensori classificati per contributo, spiegazione passo-passo per un non-tecnico
5	Human-in-the-loop controls	Percorso reale per accettare, contestare o ribaltare qualsiasi output
6	Decision log	Ogni inferenza, la sua evidenza, ogni override umano

#	Deliverable	Cosa deve esistere concretamente
7	Adaptability demonstration	Run su un secondo dominio non-sensore, o walkthrough architetturale ragionato
8	Data-flow record	Cosa esce dall'ambiente, verso quale modello, perché — più prova che il layer LLM è sostituibile

I quattro vincoli architetturali

Questi non sono deliverable, sono proprietà che il sistema deve avere ovunque. Vanno decisi prima di scrivere codice perché dopo non si aggiungono.

1. **Data minimization by design.** L'LLM ragiona solo su artefatti derivati: profili statistici, matrici di correlazione, testo delle regole. Mai righe grezze. La challenge lo ripete tre volte in tre documenti diversi.
2. **LLM swappable via configurazione.** La pipeline deve girare contro un modello locale o ospitato in UE cambiando un file di config, non riscrivendo codice. Sottinteso: AI Act, sovranità del dato, Norrin è finlandese e vende a industrie europee.
3. **Le etichette di fault non entrano nella detection.** Il CSV contiene `faultNumber` : serve solo a valutare, mai a rilevare. Usarlo nella pipeline live è barare ed è scritto esplicitamente.
4. **Portabilità progettata dall'inizio.** La logica di estrazione regole, drift e root-cause non deve essere hard-coded su questo dataset. "Not added afterward" è testuale.

La distinzione che vale un criterio intero

Un sensore morto non è un guasto di processo. La pipeline deve controllare **prima** se il dato è affidabile, e solo **dopo** ragionare sul processo. Se il dato non è affidabile deve dirlo e fermarsi, non produrre una diagnosi di fault che in realtà è un sensore congelato.

Questo è il criterio 2 su 4 della rubrica e è anche il più facile da dimostrare in demo: basta iniettare un guasto sensore finto in un batch e far vedere che il sistema dice "non mi fido di questo dato" invece di diagnosticare Fault 4.

Bonus che valgono davvero

Non sono tutti uguali. In ordine di rapporto impatto/sforzo:

- **No-egress mode** su modello open-weight locale: dimostra il vincolo 2 e il gate insieme. Alto valore, sforzo medio.
- **Step di critica** che sfida la diagnosi prima di presentarla: pochissimo codice, colpisce dritto il criterio human-in-the-loop.

- **Interfaccia in linguaggio naturale** per chiedere "perché" su un flag: se avete già il decision log strutturato, è quasi gratis.
- **Terzo dominio** (testo libero o log): solo se avanza tempo.
- **Dashboard visuale del drift**: serve comunque per la demo, quindi lo fate a prescindere.

Il dataset: cosa avete in mano

Il file `te_process.csv` pesa 6,01 GB. È il Tennessee Eastman Process, un simulatore di impianto chimico creato da Downs & Vogel nel 1993 e diventato lo standard mondiale per testare algoritmi di fault detection.

Struttura del CSV

L'intestazione che ho letto (senza aprire il resto del file) dice tutto:

```
faultNumber,simulationRun,sample,xmeas_1,...,xmeas_41,xmv_1,...,xmv_11,source
,fault_status
0.0,1.0,1,0.25038,3674.0,4529.0,...,18.447,train,normal
```

Colonna	Cosa è	Uso consentito
<code>faultNumber</code>	ID del guasto simulato, 0 = normale	Solo valutazione. Mai nella detection
<code>simulationRun</code>	ID della run di simulazione	Serve per splittare: una run è una serie temporale continua
<code>sample</code>	Indice temporale dentro la run	Il vostro asse tempo
<code>xmeas_1</code> ... <code>xmeas_41</code>	41 variabili misurate (sensori)	Input della pipeline
<code>xmv_1</code> ... <code>xmv_11</code>	11 variabili manipolate (attuatori, valvole)	Input della pipeline
<code>source</code>	train / test	Split già fatto
<code>fault_status</code>	normal / fault	Solo valutazione

Le 52 colonne `xmeas` / `xmv` sono quelle "senza etichetta" che il sistema deve interpretare da solo.

Nota strategica sui nomi. I prefissi `xmeas` e `xmv` sono un regalo: la challenge chiede esplicitamente *"a structural first pass distinguishing measured/output variables from manipulated/actuator variables"*. Ma non potete limitarvi a leggere il prefisso e dichiarare il lavoro fatto, perché quella sarebbe un'etichetta asserita senza evidenza. Dovete **derivarla dai dati** (gli attuatori hanno varianza e comportamento dinamico diversi dai misurati, e guidano i misurati con un lag) e poi mostrare che la vostra inferenza coincide col prefisso. Quella coincidenza è la vostra validazione gratuita, ed è un momento fortissimo in demo.

I fault

Il TE ha 20 modalità di guasto classiche (alcune versioni arrivano a 28). Per il vostro scopo la tassonomia utile è questa:

- **Step:** un parametro salta a un nuovo valore. Facili da rilevare.
- **Random variation:** aumenta la varianza di un ingresso. Medi.
- **Slow drift:** deriva lenta, tipicamente la cinetica di reazione. **Questo è il fault della narrativa di Norrin** — il sensore che deriva per settimane senza far scattare allarmi. Se il vostro sistema lo prende bene, avete la demo.
- **Sticking valve:** una valvola si blocca. Rilevante perché assomiglia a un sensore congelato ma non lo è.
- **Unknown:** alcuni fault non hanno causa documentata. Perfetti per mostrare incertezza onesta.

I fault 3, 9 e 15 sono notoriamente quasi impossibili da rilevare. Non inseguiteli: **dite che non li rilevate con confidenza**. La rubrica premia l'incertezza dichiarata più della sicurezza finta.

Come gestire 6 GB senza soffrire

Non caricatelo mai in pandas per intero. Tre mosse, in quest'ordine:

1. **Convertite in Parquet partizionato** per `faultNumber` e `simulationRun`. Passate da 6 GB a circa 500 MB–1 GB, con letture selettive istantanee. Va fatto una volta, all'inizio, da una persona sola.
2. **Usate DuckDB** per tutto il profiling statistico. Legge i Parquet, fa aggregazioni su miliardi di righe in memoria costante, sintassi SQL. È lo strumento giusto per questo problema e nessuno del team deve impararlo davvero.
3. **Lavorate su campioni stratificati** durante lo sviluppo: 2-3 run per fault type bastano per costruire tutto. Il run completo lo fate solo alla fine, per la demo.

Una nota che vale punti: il vincolo "i dati non escono" e il vincolo "6 GB" spingono nella stessa direzione. Un'architettura che riassume localmente e manda solo artefatti derivati è **anche** l'unica architettura che scala. Ditelo in presentazione: la privacy qui non è un

costo, è la scelta ingegneristica corretta.

La rubrica, tradotta

Quattro criteri portano il punteggio, uno è una gate. Ecco cosa significano in pratica e come si dimostrano in tre minuti di demo.

Criterio	Cosa chiedono davvero	Come lo dimostrare
1. End-to-end autonomy	Nessuna etichetta manuale, nessun prompt scritto a mano dall'utente	Lanciate il sistema su un file mai visto e uscite un report senza toccare nulla
2. Data quality	Sensore morto e processo che deriva devono leggersi diversi	Iniettate un sensore congelato: il sistema dice "dato non affidabile", non "Fault 4"
3. Human in the loop	Ogni flag risale a un'evidenza contestabile	Cliccate un alert, vedete la catena fino al numero che l'ha generato, fate override, il log lo registra
4. Adaptability	La stessa logica gira su record aziendali disordinati	Stesso codice, secondo dataset, zero modifiche al core
Gate	I dati grezzi non escono mai	Mostrate il log delle chiamate esterne: solo JSON derivati

Criterio 1 — la trappola dell'autonomia

"No hand-tuned prompts" è una condanna per l'approccio ovvio. Se scrivete un prompt che dice *"questi sono dati di un reattore chimico, xmeas_7 è la pressione"*, avete già perso il criterio. Il sistema deve **generare da sé** il prompt di analisi, partendo dai profili statistici che ha calcolato.

Regola pratica: se cambiaste dataset e doveste riscrivere un prompt, non siete autonomi.

Criterio 3 — il più sottovalutato, il più facile da vincere

Quasi tutti i team faranno un bottone "Override" che non fa niente. Il vostro deve **cambiare lo stato del sistema**: se l'operatore contesta che `xmeas_12` sia un livello di liquido, quella correzione entra nel contesto e le inferenze a valle si ricalcolano.

La frase della rubrica: *“una conclusione corretta senza derivazione mostrata deve segnare meno di una ben ragionata e appropriatamente incerta”*. Traduzione: **è permesso sbagliare, non è permesso non mostrare il perché**.

Criterio 4 — dove si spacca il campo

Qui la maggior parte dei team si fermerà a “l’architettura sarebbe portabile”. Il documento accetta il walkthrough ragionato, ma un run vero su un secondo dominio vi mette in testa.

Serve poco: un dataset transazionale sporco (ordini, fatture, log di vendita) con errori di inserimento manuale. La pipeline gira uguale perché i concetti si mappano uno a uno:

Dominio sensori	Dominio business
Colonna sensore	Campo del record
Profilo statistico	Profilo del campo (cardinalità, formato, distribuzione)
Sensore congelato	Valore default ripetuto
Fuori range	Violazione di formato o dominio
Correlazione tra sensori	Dipendenza funzionale tra campi
Drift di processo	Deriva della distribuzione nel tempo
Fault	Modalità di errore sistematico

Preparate questa tabella come slide. Da sola vale il criterio 4 anche se il run fosse imperfetto.

La gate

Non è un criterio con un punteggio, è una condizione di ammissibilità. Il modo per non perderla per distrazione: **un unico punto nel codice da cui passano tutte le chiamate al modello**, che valida il payload contro uno schema che vieta array numerici grezzi e registra tutto. Se nessuno può chiamare l’LLM aggirando quel punto, il gate è strutturalmente rispettato e potete mostrarlo come garanzia architetturale, non come promessa.

Architettura di riferimento

La pipeline ha sette stadi. Ogni stadio legge artefatti dal precedente e ne scrive di nuovi su disco. Nessuno stadio chiama un altro stadio direttamente: è questo che vi permette di lavorare in cinque senza bloccarvi.

flowchart TD

```

A[CSV 6GB<br/>zona dati grezzi] --> B[S1 Ingest<br/>Parquet + schema]
B --> C[S2 Profiling<br/>statistiche per colonna]
C --> D[S3 Relazioni<br/>correlazioni e lag]
D --> E[S4 Semantica<br/>ruolo inferito + evidenza]
B --> F[S5 Data Quality<br/>gate di affidabilit ]
F -->|dato affidabile| G[S6 Drift e Fault<br/>attribuzione per segnale]
F -->|dato rotto| H[Stop: allarme dato]
E --> G
G --> I[S7 Diagnosi<br/>critica e spiegazione]
I --> J[Operatore<br/>accetta / contesta / ribalta]
J --> K[(Decision log)]
E --> K
F --> K
G --> K

```

Il confine di fiducia   l'unica linea che conta: **i box grigi girano in locale sui dati grezzi, l'LLM vede solo quello che gli stadi producono.**

S1 — Ingest e normalizzazione

Legge il CSV, scrive Parquet partizionati, produce uno schema. Non sa nulla di sensori: vede solo colonne tipizzate e un asse temporale. Stessa identica funzione girer  sul dataset business.

Output: `schema.json` (nomi, tipi, cardinalit , range grezzi, colonna tempo individuata).

S2 — Profiling statistico

Per ogni colonna calcola la carta d'identit : media, deviazione, quantili, skewness, stazionariet , autocorrelazione, livello di rumore, numero di valori distinti, tasso di variazione, presenza di plateau. Questa   la **base di evidenza** di tutto il resto: ogni affermazione futura del sistema deve poter puntare a un numero qui dentro.

Output: `profiles.json`, una voce per colonna.

S3 — Analisi relazionale

Correlazione tra tutte le coppie, cross-correlazione con lag per trovare chi guida chi, clustering delle colonne in gruppi che si muovono insieme. Il lag   la parte preziosa: se A si muove e B lo segue con 5 campioni di ritardo, avete una relazione causale plausibile e un'evidenza mostrabile.

Output: `relations.json` (matrice correlazione, coppie con lag, cluster).

S4 — Inferenza semantica

È il primo stadio che usa l'LLM. Riceve **solo** `profiles.json` e `relations.json`, mai dati. Propone per ogni colonna: ruolo funzionale, tipo di strumento plausibile, unità di misura probabile, ed è obbligato a citare gli ID delle evidenze che ha usato. Se non può citare evidenza, deve dichiarare bassa confidenza.

Due accorgimenti che valgono punti:

- Il prompt è **generato** dal profilo, non scritto a mano. Una funzione costruisce il testo a partire dai numeri.
- Prima passata strutturale automatica (misurato vs manipolato) da varianza e comportamento dinamico, poi l'LLM lavora sopra quella. La coincidenza col prefisso `xmeas / xmv` è la vostra prova di correttezza.

Output: `semantics.json` (ruolo, confidenza, lista evidenze per colonna).

S5 — Data quality gate

Gira su ogni batch **prima** di qualsiasi ragionamento di processo. Due famiglie di check:

1. **Baseline, automatici:** completeness (buchi), validity (fuori range rispetto al profilo storico), consistency (relazioni note violate), timeliness (timestamp duplicati o mancanti), più i rilevatori di sensore congelato e saturo.
2. **Compilati da regole in linguaggio naturale:** l'operatore scrive *"la temperatura del reattore non deve superare 120"*, l'LLM la traduce in una regola eseguibile, l'operatore la vede e la approva prima che diventi attiva.

L'output non è solo pass/fail. È un **verdetto di fiducia** sul batch: affidabile, degradato, o non affidabile. Se non affidabile, la pipeline si ferma e lo dice.

Output: `dq_report.json` + stato del batch.

S6 — Drift e rilevamento fault

Due meccanismi complementari, entrambi senza mai guardare `faultNumber`:

- **Deviazione multivariata** dalla baseline normale, con il contributo per singolo segnale. Un modello classico (PCA con statistiche di controllo) va benissimo e regala l'attribuzione per costruzione: il contributo di ogni segnale allo scostamento è calcolabile. Questo soddisfa *"attribution to specific signals, not an unexplained aggregate score"*.
- **Rilevamento di deriva lenta** su finestre mobili, per il caso narrativo di Norrin: il valore resta in range ma la tendenza si sposta.

Output: `drift_events.json` (timestamp, severITÀ, sensori responsabili classificati, tipo: abrupt o gradual).

S7 — Diagnosi e critica

Prende l'evento, i sensori responsabili, i loro ruoli inferiti da S4, e costruisce una spiegazione passo-passo leggibile. Poi — e questo è il bonus più economico della lista — un secondo passaggio **critica la diagnosi**: cerca spiegazioni alternative, controlla se il data quality report suggerisce che sia un problema di dato, e abbassa la confidenza se trova ragioni per dubitare.

Output: `diagnosis.json` (tipo fault, sensori in ordine di contributo, catena di ragionamento, confidenza, critiche).

Il decision log, trasversale

Ogni stadio scrive qui, in append-only. Una riga per decisione: cosa, quando, su quale evidenza, con quale confidenza, quale modello è stato chiamato, e ogni override umano con la sua motivazione. Non è un file di log: è un deliverable e sarà guardato dai giudici. Progettatelo come tale.

Contratti di interfaccia

Questa è la sezione che decide se il vostro hackathon finisce bene. In cinque persone, il fallimento tipico non è il codice sbagliato: è che alle 2 di notte tre moduli funzionanti non si parlano.

La regola: nella prima ora scrivete gli schemi JSON, non il codice. Ognuno poi implementa contro lo schema e produce subito un file finto conforme. Chi sta a valle lavora sul file finto e non aspetta nessuno.

L'artefatto atomico: l'evidenza

Tutto il sistema ruota attorno a un unico oggetto. Se sbagliate questo, la tracciabilità non c'è e perdete il criterio 3.

```
{
  "evidence_id": "ev_0042",
  "stage": "S3_relations",
  "kind": "lagged_correlation",
  "subject": ["col_12", "col_31"],
  "value": {"corr": 0.91, "lag_samples": 5},
  "computed_at": "2026-09-19T10:14:00Z",
  "method": "pearson_cross_correlation",
  "window": {"run": 3, "from": 0, "to": 500}
}
```

Ogni inferenza, flag e diagnosi porta una lista di `evidence_id`. In demo, cliccare un alert e risalire fino a questo oggetto è letteralmente il criterio 3 che si dimostra da solo.

Gli artefatti di stadio

Artefatto	Prodotto da	Consumato da	Chiave
<code>schema.json</code>	S1	S2, S5	<code>columns[]</code> con nome, tipo, ruolo temporale
<code>profiles.json</code>	S2	S3, S4, S5, S6	<code>col_id</code> → statistiche + <code>evidence_ids</code>
<code>relations.json</code>	S3	S4, S6	coppie, lag, cluster + <code>evidence_ids</code>
<code>semantics.json</code>	S4	S7, UI	<code>col_id</code> → ruolo, confidenza, <code>evidence_ids</code> , <code>human_override</code>
<code>dq_report.json</code>	S5	S6, S7, UI	check, esito, <code>trust_verdict</code> del batch
<code>drift_events.json</code>	S6	S7, UI	evento, severità, <code>ranked_signals[]</code> , <code>evidence_ids</code>
<code>diagnosis.json</code>	S7	UI	fault, sensori, catena, confidenza, critiche
<code>decision_log.jsonl</code>	tutti	UI, giudici	append-only, una riga per decisione

La regola dell'ID di colonna

Nessuno stadio a valle usa mai il nome originale della colonna. Tutti usano `col_id` opaco assegnato da S1. Motivo: il giorno in cui girate sul dataset business, i nomi cambiano ma il codice no. È una riga di disciplina che vi regala metà del criterio 4.

Il guardiano del gate

Un solo modulo può parlare con un LLM. Firma unica, usata da tutti:

```
def call_model(purpose: str, payload: dict, schema_out: dict) -> dict:
    """Unico varco verso l'esterno.
    - valida payload contro un allow-list di chiavi derivate
    - rifiuta array numerici grezzi e qualsiasi cosa somigli a righe
    - registra: cosa, a quale modello, perche, quando, quanti token
    - il modello viene da config, non da codice
    """
```

Se questa funzione rifiuta il payload, lo stadio deve riassumere di più. Il gate diventa così una proprietà del sistema, non una promessa in slide. E il suo log di chiamate è il deliverable 8, già pronto.

Config unica per il modello

```
llm:
  provider: local          # local | eu_hosted | anthropic | google
  model: llama-3.1-8b
  endpoint: http://localhost:11434
  max_payload_kb: 64
```

Cambiare `provider` e rilanciare, davanti ai giudici, è la dimostrazione richiesta dal deliverable 8. Prevedete di avere Ollama installato su almeno una macchina, anche con un modello piccolo: il "no-egress mode" è uno dei bonus più pagati.

Le tre regole di ingaggio del team

1. **Nessuno importa il modulo di un altro.** Si leggono solo artefatti da disco. Così un modulo rotto non blocca gli altri.
2. **Ogni stadio ha un file di esempio conforme, committato subito.** Prima del codice vero.
3. **Ogni stadio si lancia da solo** con un comando singolo su un campione. Se serve tutta la pipeline per testare un pezzo, avete già perso tempo.

Divisione dei compiti

Cinque ruoli, ognuno proprietario di artefatti specifici. Il principio: **ogni persona possiede un file di output, non una cartella di codice.** Così la responsabilità è verificabile.

Ruolo	Possiede	Deliverable che copre	Dipende da
A — Data & Infra	schema.json , profiles.json , Parquet	1 (base di evidenza)	Nessuno: parte subito
B — Relazioni & Semantica	relations.json , semantics.json	1	Profili di A
C — Data Quality	dq_report.json , compilatore regole	2	Schema di A
D — Drift & Diagnosi	drift_events.json , diagnosis.json	3, 4	Profili di A, semantica di B
E — Trust Layer & UI	decision_log.jsonl , call_model , dashboard	5, 6, 8	Gli schemi, non il codice

Il deliverable 7 (adaptability) è **di tutti**: nessuno hard-codea nulla. La verifica finale la fa E lanciando la pipeline sul secondo dataset.

Chi fa cosa, in concreto

A — Data & Infra. È il collo di bottiglia iniziale, quindi parte per primo e da solo. Conversione Parquet, setup DuckDB, campione stratificato per lo sviluppo, poi il profiler statistico. Deve consegnare `profiles.json` su un campione **entro le prime tre ore**, altrimenti B e D restano fermi. Dopo, diventa la persona che ottimizza e prepara il run completo per la demo.

B — Relazioni & Semantica. La parte più intellettualmente interessante e la più visibile in demo. Correlazioni con lag, clustering, poi la generazione automatica del prompt dai profili. Attenzione: il suo lavoro è quello più a rischio di scivolare nel "prompt scritto a mano". Deve poter dire ai giudici "questo prompt è generato da questa funzione".

C — Data Quality. Il ruolo più sottovalutato ed è un criterio intero della rubrica. I check baseline sono codice semplice e scrivibile prima ancora di avere i dati. La parte con impatto è il compilatore di regole: regola in italiano → funzione eseguibile → approvazione umana. C deve anche preparare **l'iniettore di guasti sensore** per la demo: è lui che dimostra il criterio 2.

D — Drift & Diagnosi. PCA con statistiche di controllo per l'attribuzione per segnale, rilevatore di deriva lenta, poi la costruzione della spiegazione passo-passo e lo step di critica. Regola ferrea per D: `faultNumber` esiste solo nel suo script di valutazione, mai nel codice della pipeline. Tenga i due file fisicamente separati.

E — Trust Layer & UI. Il ruolo più importante è quello che la maggior parte dei team assegna male. E non aspetta nessuno: costruisce subito `call_model`, il decision log e la UI contro gli artefatti finti. Quando i dati veri arrivano, l'interfaccia è già pronta. E è anche chi presenta, perché è l'unico che ha visto tutto il sistema dall'alto.

Chi mettere dove

Non sapete il dominio, quindi distribuite sulla base di cosa è più rischioso lasciare a chi è meno a suo agio:

- Se qualcuno è forte in Python/dati: **A** o **D**.
- Se qualcuno è forte in frontend o sa far vedere le cose: **E**, senza discussione.
- Se qualcuno è alle prime armi: **C**. I check baseline sono il punto di ingresso più gentile del progetto e valgono comunque un criterio intero.
- Tu, che stai coordinando e hai il quadro d'insieme: **B** o **E**. Se prendi E presenti tu, e con questo documento in mano hai il contesto per farlo.

La regola anti-caos

Un solo repository, un branch per persona, merge su main solo ai checkpoint di integrazione. Niente review: non c'è tempo. La protezione non è la review, sono gli schemi JSON fissati all'inizio.

Workflow e checkpoint

Il piano assume un hackathon di circa 36 ore. Adattate le ore, non l'ordine: la sequenza è progettata perché nessuno resti mai bloccato ad aspettare.

Fase	Durata	Obiettivo	Fatto quando
0. Contratti	1 h, tutti insieme	Schemi JSON e repo	Ogni artefatto ha un file di esempio committato
1. Sviluppo cieco	4 h, in parallelo	Ogni stadio gira sui file finti	Cinque moduli girano isolati
2. Primo innesto	1 h, tutti	Artefatti veri di A sostituiscono i finti	La catena $S1 \rightarrow S2 \rightarrow S4$ produce output reale
3. Costruzione	8 h, in parallelo	Ogni stadio sul dato vero	La pipeline gira end-to-end una volta
4. Integrazione	2 h, tutti	Un solo comando, dall'inizio alla fine	<code>python run.py --config local.yaml</code> funziona

Fase	Durata	Obiettivo	Fatto quando
5. Adaptability	3 h, D+E	Run sul secondo dataset	La pipeline produce un report sul dataset business
6. Demo prep	4 h, tutti	Script della presentazione provato	Provata due volte cronometro alla mano
Riserva	il resto	Bonus e imprevisti	—

Fase 0: l'ora che decide tutto

Non scrivete codice. Fate queste sei cose, in ordine:

1. Aprite un repo, un solo file `contracts/` con gli schemi JSON di questo documento.
2. Ogni persona crea il file di esempio del suo artefatto, riempito a mano con valori finti plausibili.
3. Decidete i `col_id: col_000 ... col_051`, mappati in `schema.json`. Nessuno usa mai `xmeas_7` nel codice.
4. Scrivete la firma di `call_model` e committatela. Nessuno chiama un LLM in altro modo.
5. Scegliete il secondo dataset **adesso**. Non alla fine. (Suggerimento: un dump di ordini o transazioni con errori di data entry, si trovano su Kaggle in cinque minuti.)
6. Scrivete su una lavagna gli otto deliverable e chi li possiede. Restano visibili tutto il tempo.

I checkpoint di integrazione

Tre momenti in cui tutti si fermano e la pipeline deve girare intera, anche male:

- **Checkpoint 1 (fine fase 2):** la catena gira su artefatti misti veri/finti. Serve a scoprire i disallineamenti di schema quando costano dieci minuti, non tre ore.
- **Checkpoint 2 (fine fase 4):** un comando, dall'inizio alla fine, sui dati veri. Da qui in poi **main deve sempre girare**. Chi rompe main lo ripara subito.
- **Checkpoint 3 (metà fase 6):** demo completa a freddo, cronometrata, con una persona che fa il giudice ostile e chiede "perché questo numero?".

Lo script della demo

Provatelo. La differenza tra un buon progetto e una vittoria è quasi sempre qui. Sequenza suggerita, cinque minuti:

1. **Il problema in una frase**, con il grafico del sensore che deriva dentro il range. Trenta secondi.

2. **Lancio a freddo** su dati mai visti, nessuna etichetta, nessun prompt digitato. Esce il sensor report.
3. **Il momento validazione**: "il sistema ha dedotto dai dati quali colonne sono attuatori, senza saperlo. Ecco la conferma."
4. **Il momento data quality**: iniettate un sensore congelato. Il sistema dice "non mi fido di questo batch" invece di diagnosticare un fault. Questa è la scena che ricorderanno.
5. **Diagnosi con catena di evidenza**: cliccate il flag, risalite fino al numero. Poi l'override: l'operatore contesta, il sistema aggiorna e il log registra.
6. **Il secondo dominio**: stesso comando, dataset di fatture, esce un report. Nessuna riga di codice cambiata.
7. **Il gate**: mostrate il log delle chiamate esterne e cambiate `provider: local`. Rilanciate. Funziona.

Chiudete con la frase che la challenge stessa usa: un sistema che si guadagna la fiducia di un operatore, non uno che asserisce conclusioni che non sa mostrare.

Trappole

Otto modi documentati di perdere questa challenge. I primi tre affondano più team di tutti gli altri messi insieme.

1. **Il prompt scritto a mano**. Qualcuno, verso le due di notte, scriverà nel prompt *"questi dati vengono da un processo chimico con un reattore e uno stripper"* perché migliora i risultati. Migliora i risultati e distrugge il criterio 1. Mettete un test: se il prompt contiene una parola di dominio non derivata dai dati, fallisce la build.
2. **La fuga del dato grezzo**. Un `df.head().to_string()` dentro un prompt per fare debug, dimenticato lì. È la gate, ed è esattamente così che si perde per distrazione. Il guardiano `call_model` esiste per questo: non deve essere possibile aggirarlo, nemmeno per pigrizia alle tre di notte.
3. **faultNumber che filtra nella pipeline**. Facilissimo: si usa per valutare, poi qualcuno lo passa a una funzione per comodità. Difesa strutturale: S1 scrive **due** Parquet separati, uno con i dati e uno con le etichette, e il modulo di detection non ha nemmeno il path del secondo.
4. **Le prime tre ore spese a guardare il CSV**. Sei gigabyte invitano all'esplorazione infinita. A lavora su un campione stratificato di poche run e basta. Il file completo si tocca solo per il run finale.
5. **La dashboard che mangia tutto**. E passerà due ore a far stare bene un grafico. La UI deve essere brutta e funzionante: una lista di flag, una catena di evidenza cliccabile, un bottone override che cambia davvero lo stato. Nient'altro conta.

6. L'adaptability rimandata. Se il secondo dominio arriva alle ultime tre ore, non arriva. Sceglietelo in fase 0 e fate girare S1+S5 su quello già a metà hackathon, anche solo per vedere che non esplode.

7. L'override finto. Un bottone che scrive su un log e non cambia niente. I giudici lo chiederanno: "cosa succede se contesto questa inferenza?". La risposta deve essere una modifica di stato visibile, non un messaggio di conferma.

8. Nessuno prova la presentazione. Cinque persone che hanno costruito cinque pezzi non sanno raccontare un sistema. Serve una persona che prova ad alta voce, due volte, col cronometro.

Rischi tecnici e la loro mitigazione

Rischio	Segnale precoce	Mitigazione
Il profiling non scala	Una query di test impiega minuti	DuckDB su Parquet, mai pandas sul CSV
L'LLM inventa ruoli sensori	Confidenze tutte alte, evidenze vaghe	Output strutturato obbligatorio con <code>evidence_ids</code> ; niente evidenza, niente inferenza
PCA rileva tutto come anomalia	Falsi positivi ovunque	Baseline solo su <code>faultNumber = 0</code> , soglie dai quantili della baseline
I moduli non si integrano	Errori di schema al checkpoint 1	Schemi fissati in fase 0, file di esempio da subito
Il modello locale non gira	Ollama non installato quando serve	Installatelo in fase 0 su una macchina, modello piccolo, testato

Una cosa da non dimenticare

La challenge dice che qualcuno potrebbe riconoscere il dataset e trovare materiale sul processo industriale, e che *"quella conoscenza può informare un'ipotesi, ma non sostituisce l'evidenza"*.

Leggetela come un permesso condizionato: potete usare quello che sapete sul Tennessee Eastman per **generare ipotesi**, purché il sistema poi le verifichi sui dati e mostri il risultato. Un'ipotesi da conoscenza di dominio, confermata o smentita da un'evidenza calcolata, è esattamente ciò che la challenge chiede. Un'etichetta copiata da un paper non lo è.